데이터분석을위한 PANDAS

DATA PREPROCESSING

데이터 전처리

데이터 전처리

◆ 전처리(Preprocessing)

데이터 품질을 높이기 위한 과정
분석 목적에 맞게 변형하는 과정
많은 시간 소요, 필수 과정

- **데이터 처리** => 누락 데이터, 중복 데이터, 이상 데이터
- **데이터 변환** => 단위 표준화, 자료형 표준화
- **데이터 크기** => 정규화

전처리 – 결측 데이터

데이터 전처리

◆ 결측 데이터 처리

- 결측치 데이터

  - 측정되지 않았거나 **누락된 값, 입력되지 않은 값**
  - 분석 **정확성 떨어짐 / 분석 결과 왜곡** 될 수 있음

  - 처리 방법 : 제거, 치환 & 대체, 특별한 범주 간주

데이터 전처리

◆ 결측 데이터 처리

❖ 표기방식

→ numpy.NaN	Not a Number 약자.	수치 결측값
→ pandas.NaT	Not a Time 약자.	시간/날짜 결측값
→ pandas.NA	Not Available.	일반적인 누락 데이터
→ None		파이썬에서 누락 데이터
→ null		NA와 동일 의미

◆ 결측 데이터 처리

- 관련 메서드

결측 데이터 체크

→ bool = 객체.**isnull( )** : null이면 True

→ bool = 객체.**isna( )** : na이면 True

정상 데이터 체크

→ bool = 객체.**notnull( )** : null이면 False

→ bool = 객체.**notna( )** : na이면 False

◆ 결측 데이터 처리

- 체크 메서드

```
객체.isnull( )
```

	survived	pclass	sex	age	...	deck	embark_town	alive	alone
0	False	False	False	False	...	True	False	False	False
1	False	False	False	False	...	False	False	False	False
2	False	False	False	False	...	True	False	False	False
3	False	False	False	False	...	False	False	False	False
4	False	False	False	False	...	True	False	False	False

데이터 전처리

◆ 결측 데이터 처리

- 체크 메서드

객체.isnull( ).sum( ) False 0, True 1로 계산

	survived	pclass	sex	age	...	deck	embark_town	alive	alone
0	False	False	False	False	...	True	False	False	False
1	False	False	False	False	...	False	False	False	False

	survived	pclass	sex	age	...	deck	embark_town	alive	alone
0	0	0	0	177	...	688	2	0	0

◆ 결측 데이터 처리

- 체크 메서드

```
## 모듈 로딩
import pandas as pd
import numpy as np

## DF 생성
df = pd.DataFrame(dict( age  =[ 5, 6, np.nan ],
                        born =[ pd.NaT, pd.Timestamp('1939-05-27'),
                                pd.Timestamp('1940-04-25')],
                        name=['Alfred', 'Batman', ''],
                         toy=[None, 'Batmobile', 'Joker']) )

## DF 출력
print( df )
```

	age	born	name	toy
0	5.0	NaT	Alfred	None
1	6.0	1939-05-27	Batman	Batmobile
2	NaN	1940-04-25		Joker

◆ 결측 데이터 처리

- 체크 메서드

```
## 결측치 체크
df.isna( )
```

	age	born	name	toy
0	False	True	False	True
1	False	False	False	False
2	True	False	False	False

	age	born	name	toy
0	5.0	NaT	Alfred	None
1	6.0	1939-05-27	Batman	Batmobile
2	NaN	1940-04-25		Joker

◆ 결측 데이터 처리

- 처리 방법 ① 제거

객체.**dropna( )**	**결측값 존재 행/열 삭제**

- 매개변수(Parameter)
  - → axis = 0 행(row) 삭제
  - → axis = 1 열(column) 삭제
  - → how = 'all' 모든 값 결측치인 경우 삭제
  - → how = 'any' 하나라도 결측치인 경우 삭제 (기본)
  - → thresh = 숫자 Non-NA개 이상, 임계치 설정
  - → subset = 조건 하위 조건

데이터 전처리

◆ 결측 데이터 처리

- 처리 방법 ① 제거

```
## 모듈 로딩
import pandas as pd
import numpy as np

## DF 생성
df = pd.DataFrame({"name": ['Alfred', 'Batman', 'Catwoman'],
                   "toy"  : [np.nan, 'Batmobile', 'Bullwhip'],
                   "born" : [pd.NaT, pd.Timestamp("1940-04-25"),
                             pd.NaT]})

## DF 출력
print( df )
```

```
       name        toy       born
0    Alfred        NaN        NaT
1    Batman  Batmobile 1940-04-25
2  Catwoman   Bullwhip        NaT
```

◆ 결측 데이터 처리

- 처리 방법 ① 제거

```
## axis : 결측치 검사를 기준 축 설정
##        0, 'index'   – 행
##        1, 'columns' – 열
df.dropna(axis='index')   <= 기본값
```

	name	toy	born
0	Alfred	NaN	NaT
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

	name	toy	born
1	Batman	Batmobile	1940-04-25

```
df.dropna( axis='columns' )
```

	name	toy	born
0	Alfred	NaN	NaT
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

	name
0	Alfred
1	Batman
2	Catwoman

◆ 결측 데이터 처리

- 처리 방법 ① 제거

```
## how : 제거 방식/기준 설정
##        'all'   - 모든 속성이 결측치인 행
##        'any' - 1개 이상 속성이 결측치인 행
df.dropna( how='all' )
```

	name	toy	born
0	Alfred	NaN	NaT
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

	name	toy	born
0	Alfred	NaN	NaT
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

◆ 결측 데이터 처리

- 처리 방법 ① 제거

```
## thresh : 결측치가 아닌 데이터  최소 개수 설정
##          how와 함께 사용 불가
df.dropna( thresh=2 )
```

	name	toy	born
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

```
df.dropna( thresh=1 )
```

	name	toy	born
0	Alfred	NaN	NaT
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

◆ 결측 데이터 처리

- 처리 방법 ① 제거

```
## subset : 특정 컬럼 또는 행만 결측치 검사하도록 설정

df.dropna( subset=['name']  )
```

	name	toy	born
0	Alfred	NaN	NaT
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

```
df.dropna( subset=['name‘, ‘toy’] )
```

	name	toy	born
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

◆ 결측 데이터 처리

- 처리 방법 ② 치환/대체

객체.**fillna(** 치환값 **)** **특정값, 평균값, 최빈값으로 치환**

- 매개변수(Parameter) : **value**
  - → fillna(0) : 모두 동일 값
  - → fillna({ 컬럼: 0, 컬럼: 1, … , 컬럼: N }) : 컬럼별 다른 값
  - → fillna(평균값)
  - → fillna(최빈값)
  - → fillna(중앙값)

◆ 결측 데이터 처리

- 처리 방법 ② 치환/대체

객체.**fillna(** 치환값 **)** **특정값, 평균값, 최빈값으로 치환**

- 매개변수(Parameter) : **method**
  - → 'ffill' 직전 행(row) 값으로 치환
  - → 'bfill' / 'backfill' 바로 다음 행(row) 값으로 치환

- 매개변수(Parameter) : **limit**
  - → int 지정된 수 만큼 치환
  - → None 모두 치환

◆ 결측 데이터 처리

- 처리 방법 ② 치환/대체

```
## 모듈 로딩
import pandas as pd
import numpy as np

## DF 생성
df = pd.DataFrame( [ [np.nan, 2, np.nan, 0],
                     [3, 4, np.nan, 1],
                     [np.nan, np.nan, np.nan, np.nan],
                     [np.nan, 3, np.nan, 4]],
                     columns=list("ABCD"))

## DF 출력
print( df )
```

	A	B	C	D
0	NaN	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	NaN	NaN	NaN	NaN
3	NaN	3.0	NaN	4.0

◆ 결측 데이터 처리

- 처리 방법 ② 치환/대체

```
## value : NaN를 대체할 값 설정
## 모든 NaN 동일 값으로 채우기
df.fillna(value=0 )
```

	A	B	C	D
0	NaN	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	NaN	NaN	NaN	NaN
3	NaN	3.0	NaN	4.0

	A	B	C	D
0	0.0	2.0	0.0	0.0
1	3.0	4.0	0.0	1.0
2	0.0	0.0	0.0	0.0
3	0.0	3.0	0.0	4.0

```
## 컬럼별 NaN 대체값 설정
values = {"A": 0, "B": 1, "C": 2, "D": 3}
df.fillna( value=values )
```

	A	B	C	D
0	NaN	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	NaN	NaN	NaN	NaN
3	NaN	3.0	NaN	4.0

	A	B	C	D
0	0.0	2.0	2.0	0.0
1	3.0	4.0	2.0	1.0
2	0.0	1.0	2.0	3.0
3	0.0	3.0	2.0	4.0

◆ 결측 데이터 처리

- 처리 방법 ② 치환/대체

```
## value  : NaN를 대체할 값 설정
## limit  : 대체값 채울 개수 설정

df.fillna( value=values, limit=1 )
```

	A	B	C	D
0	NaN	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	NaN	NaN	NaN	NaN
3	NaN	3.0	NaN	4.0

	A	B	C	D
0	0.0	2.0	2.0	0.0
1	3.0	4.0	NaN	1.0
2	NaN	1.0	NaN	3.0
3	NaN	3.0	NaN	4.0

◆ 결측 데이터 처리

- 처리 방법 ② 치환/대체

```
## method : 이전 값들로 채우기 설정
##          'ffill'          - 이전 값으로 채우기
##          'backfill','bfill' - 다음 값으로 채우기
df.fillna( method='ffill' )
```

	A	B	C	D
0	NaN	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	NaN	NaN	NaN	NaN
3	NaN	3.0	NaN	4.0

	A	B	C	D
0	NaN	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	3.0	4.0	NaN	1.0
3	3.0	3.0	NaN	4.0

```
df.fillna( method='backfill' )
```

	A	B	C	D
0	NaN	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	NaN	NaN	NaN	NaN
3	NaN	3.0	NaN	4.0

	A	B	C	D
0	3.0	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	NaN	3.0	NaN	4.0
3	NaN	3.0	NaN	4.0

데이터 전처리

◆ 결측 데이터 처리

- 처리 방법 ② 치환/대체

객체.**interpolate(** 보간법 **)**

❖ 보간법

두 개 이상의 알고 있는 데이터 포인트들 사이에 예상되는 값들 추정

❖ 적용 예

→ 데이터가 누락되었을 때

→ 시계열 데이터에서 연속적인 추세 유지하고 싶을 때

→ 그래프, 곡선, 음성 신호 등에서 매끄러운 변화 표현이 필요할 때

→ 머신러닝 학습 시 결측값 처리 위한 전처리

◆ 결측 데이터 처리

- 처리 방법 ② 치환/대체

객체.**interpolate(** 보간법 **)**

❖ 매개변수 method

→ 'linear' 선형 보간. 인접한 두 값 사이의 직선 사용

→ 'time' 시간 기반 선형 보간. 날짜/시간 인덱스 사용

→ 'index' 인덱스 값 기반의 선형 보간

→ 'values' 배열의 값에 따라 선형 보간

→ 'nearest' 가장 가까운 값으로 채움

→ 그 외 다양한 보간 알고리즘 메서드 제공

데이터 전처리 - 중복데이터

◆ 중복 데이터 처리

- 중복 데이터

- **하나의 행(row)**은 **모든 속성값 존재하는 완벽한 데이터셋**
- **모든 속성값 동일한 행/열 ==> 중복 데이터**

- 검사 함수 : 객체.**duplicated( )** 중복이면 True / 아니면 False
- 제거 함수 : 객체.**drop_duplicates( )** 중복 행 제거

◆ 중복 데이터 처리

- 체크 메서드

```
객체.duplicated( )
```

- 매개변수(Parameter)
  - → subset = None　　중복 체크 위한 부분 설정 지정
  - → keep = 'first'　　중복 데이터에서 첫 번째 행 유지

◆ 중복 데이터 처리

- 체크 메서드

```
## 모듈로딩
import pandas as pd

## 데이터 준비
df = pd.DataFrame( { 'brand' : [ 'Yum', 'Yum', 'Indo', 'Indo', 'Indo' ],
                     'style'  : [ 'cup', 'cup', 'cup', 'pack', 'pack' ],
                     'rating' : [4, 4, 3.5, 15, 5] })

## 중복체크
result = df.duplicated().sum()

print(f'중복 데이터/행 : {result}개')
```

	brand	style	rating
0	Yum	cup	4.0
1	Yum	cup	4.0
2	Indo	cup	3.5
3	Indo	pack	15.0
4	Indo	pack	5.0

중복

◆ 중복 데이터 처리

- 체크 메서드

```
## 중복체크 - 첫 번째 유지, 나머지 중복
result = df.duplicated( )
```

```
0    False
1     True
2    False
3    False
4    False
dtype: bool
```

```
## 중복체크 - 마지막 유지, 그 외 중복
result = df.duplicated( keep='last' )
```

```
0     True
1    False
2    False
3    False
4    False
dtype: bool
```

```
## 중복체크 - 모두 중복
result = df.duplicated( keep=False )
```

```
0     True
1     True
2    False
3    False
4    False
dtype: bool
```

◆ 중복 데이터 처리

- 체크 메서드

```
## 중복체크 - brand 컬럼만 중복인 행
result = df.duplicated( subset='brand' )
```

		brand	style	rating
중복	0	Yum	cup	4.0
	1	Yum	cup	4.0
중복	2	Indo	cup	3.5
	3	Indo	pack	15.0
	4	Indo	pack	5.0

```
0     False
1      True
2     False
3      True
4      True
dtype: bool
```

first

first

◆ 중복 데이터 처리

- 체크 메서드

```
## 중복체크 - brand, style 컬럼이 중복인 행
result = df.duplicated( subset=['brand','style'] )
```

		brand	style	rating
중복	0	Yum	cup	4.0
	1	Yum	cup	4.0
	2	Indo	cup	3.5
중복	3	Indo	pack	15.0
	4	Indo	pack	5.0

0	False	first
1	True	
2	False	
3	False	first
4	True	
dtype: bool		

◆ 중복 데이터 처리

- 처리 방법 ① 제거

객체. **drop_duplicates( )**

- 매개변수(Parameter)
  - → subset = None 삭제 위한 부분 설정 지정
  - → keep = 'first' 중복 데이터에서 첫 번째 행 유지
  - → inplace = False 원본 데이터 유지

◆ 중복 데이터 처리

- 처리 방법 ① 제거

```
## 모듈로딩
import pandas as pd

## 데이터 준비
df = pd.DataFrame( { 'brand' : [ 'Yum', 'Yum', 'Indo', 'Indo', 'Indo' ],
                     'style'  : [ 'cup', 'cup', 'cup', 'pack', 'pack' ],
                     'rating' : [4, 4, 3.5, 15, 5] })

## 중복체크
result = df.duplicated().sum()

print(f'중복 데이터/행 : {result}개')
```

	brand	style	rating
0	Yum	cup	4.0
1	Yum	cup	4.0
2	Indo	cup	3.5
3	Indo	pack	15.0
4	Indo	pack	5.0

◆ 중복 데이터 처리

- 처리 방법 ① 제거

```
## 중복데이터 삭제 - 모든 속성값이 동일한 행 비교
df.drop_duplicates( keep= 'first' )
```

중복

	brand	style	rating
0	Yum	cup	4.0
1	Yum	cup	4.0
2	Indo	cup	3.5
3	Indo	pack	15.0
4	Indo	pack	5.0

첫번째 유지

	brand	style	rating	
0	Yum	cup	4.0	first
2	Indo	cup	3.5	
3	Indo	pack	15.0	
4	Indo	pack	5.0	

◆ 중복 데이터 처리

- 처리 방법 ① 제거

```
## 중복데이터 삭제 - 모든 속성값이 동일한 행 비교
df.drop_duplicates( keep= 'last' )
```

중복

	brand	style	rating
0	Yum	cup	4.0
1	Yum	cup	4.0
2	Indo	cup	3.5
3	Indo	pack	15.0
4	Indo	pack	5.0

마지막 유지

	brand	style	rating
1	Yum	cup	4.0
2	Indo	cup	3.5
3	Indo	pack	15.0
4	Indo	pack	5.0

last

◆ 중복 데이터 처리

- 처리 방법 ① 제거

```
## 중복데이터 삭제 - 모든 속성값이 동일한 행 비교
df.drop_duplicates( keep=False )
```

중복

	brand	style	rating
0	Yum	cup	4.0
1	Yum	cup	4.0
2	Indo	cup	3.5
3	Indo	pack	15.0
4	Indo	pack	5.0

모두 삭제

	brand	style	rating
2	Indo	cup	3.5
3	Indo	pack	15.0
4	Indo	pack	5.0

◆ 중복 데이터 처리

- 처리 방법 ① 제거

```
## 중복데이터 삭제 - brand 컬럼만 동일한 행 비교
df.drop_duplicates( subset='brand' )
```

		brand	style	rating
중복	0	Yum	cup	4.0
	1	Yum	cup	4.0
중복	2	Indo	cup	3.5
	3	Indo	pack	15.0
	4	Indo	pack	5.0

	brand	style	rating	
0	Yum	cup	4.0	first
2	Indo	cup	3.5	

◆ 중복 데이터 처리

- 처리 방법 ① 제거

```
## 중복데이터 삭제 - brand 컬럼만 동일한 행 비교
df.drop_duplicates( subset=['brand','style'] )
```

		brand	style	rating
중복	0	Yum	cup	4.0
	1	Yum	cup	4.0
	2	Indo	cup	3.5
중복	3	Indo	pack	15.0
	4	Indo	pack	5.0

	brand	style	rating	
0	Yum	cup	4.0	first
2	Indo	cup	3.5	
3	Indo	pack	15.0	first

데이터 전처리 - 형변환

데이터 전처리

◆ 데이터 표준화 및 변환

- **데이터 표준화: 형식·단위·표현 통일**

  → 수집·정리된 데이터를 동일한 형식으로 맞추는 작업

  → 날짜, 단위, 범주값, 문자열 표현 등을 일관되게 통일

  → 데이터 분석 시 오류를 줄이고 정확도를 높임

데이터 전처리

◆ 데이터 표준화 및 변환

- **자료형 변환**

  문자열, 숫자, 날짜 등 **데이터 타입을 분석 목적에 맞게 바꾸는 작업**

- **인코딩**

  문자형 또는 범주형 데이터를 **숫자 형태로 바꾸는 작업**

- **단위 변환**

  서로 다른 단위로 저장된 값을 **하나의 기준 단위로 맞추는 작업**

- **범위 변환**

  숫자 값의 범위를 0~1, -1~1 등 **일정한 기준 범위로 맞추는 작업**

데이터 전처리

◆ 자료형 변환

- 데이터 분석 하기 전 컬럼의 자료형을 목적에 맞게 바꾸는 작업

→ 숫자처럼 보이지만 >>> 문자형으로 저장된 경우 : 계산 불가

→ 날짜처럼 보이지만 >>> 문자형으로 저장된 경우 : 날짜 연산 불가

→ **분석 목적에 맞게 int, float, str, datetime 형식으로 변환 필요**

◆ 자료형 변환

- 변환 메서드

기본 자료형 => int, float, str, object

데이터 자료형 => 범주형, 연속형

❖ 메서드

객체.**astype(** 자료형 **)** => 자료형으로 변환

객체.**dtypes** => 자료형 확인

◆ 자료형 변환

- 기본 자료형 ⇔ 데이터 자료형

DataFrame.**astype(** 변환_자료형 **)**

Series.**astype(** 변환_자료형 **)**

→ dtype : 변환할 자료형 지정

→ copy : 변환할 때 원본 데이터를 복사할지 여부를 지정

→ errors : 변환 중 오류가 발생했을 때 처리 방식

- "raise" : 오류 발생 - "ignore" : 오류 무시 후 원본 반환

데이터 전처리

◆ 자료형 변환

- 기본 자료형 ⇔ 데이터 자료형

```
# 샘플 데이터 생성
df = pd.DataFrame({
    "이름": ["김민수", "이서연", "박지훈"],
    "나이": ["20", "21", "19"],
    "점수": ["85.5", "90.0", "78.5"],
    "가입일": ["2024-01-10", "2024-02-15", "2024-03-01"]
})


## 컬럼별 처리 후 저장

df["나이"] = df["나이"].astype(int)

df["점수"] = df["점수"].astype(float)
```

◆ 자료형 변환

- 기본 자료형 ⇔ 데이터 자료형

```
## 여러 컬럼의 자료형을 한 번에 처리 후 저장
df = df.astype({ "나이": int,
                 "점수": float,
                 "가입일": "datetime64[ns]"})
```

◆ 자료형 변환

- 기본 자료형 ⇔ 데이터 자료형

구분	일반 결측치 처리 불가	Nullable 결측치 처리 가능	
정수형	int int8 int64	“Int8” “Int16” “Int32” “Int64”	pd.Int8Dtype( ) pd.Int16Dtype( ) pd.Int32Dtype( ) pd.Int64Dtype( )
실수형	float float32 float64	“Float32” “Float64”	pd.Float32Dtype pd.Float64Dtype
논리형	bool	“boolean”	pd.BooleanDtype( )
문자열형	object str	“string”	pd.StringDtype( )

◆ 자료형 변환

- 기본 자료형 ⇔ 데이터 자료형

```
# 샘플 데이터 생성
df = pd.DataFrame({
    "이름": ["김민수", "이서연", "박지훈"],
    "나이": ["20", "21", "19"],
    "점수": ["85.5", "90.0", "78.5"],
    "가입일": ["2024-01-10", "2024-02-15", "2024-03-01"]
})


## 컬럼별 처리 후 저장

df["나이"] = df["나이"].astype("Int8")

df["점수"] = df["점수"].astype(“Float32”)
```

◆ 자료형 변환

- 기본 자료형 ⇔ 데이터 자료형

```
## 컬럼별 처리 후 저장
df["나이"] = df["나이"].astype(pd.Int8Dtype())
df["점수"] = df["점수"].astype(pd.Float32Dtype())
```

데이터 전처리

◆ 자료형 변환

- 문자형 컬럼 → 숫자형 int64, float64 변환

pandas.**to_numeric(** 데이터 **)**

→ downcast : 변환된 숫자 자료형 크기 축소 시 사용
'integer', 'float', 'unsigned'

→ arg : 변환할 데이터. Series, 리스트, 배열, 컬럼 데이터

→ errors : 숫자로 변환할 수 없는 값이 있을 때 처리 설정
'raise' 오류 발생, 'coerce' 변환 불가 NaN 처리

◆ 자료형 변환

- 문자형 컬럼 → 숫자형 int64, float64 변환

```
# 샘플 데이터 생성
df = pd.DataFrame({
    "이름": ["김민수", "이서연", "박지훈"],
    "나이": ["20", "21", "19"],
    "점수": ["85.5", "90.0", "78.5"],
    "가입일": ["2024-01-10", "2024-02-15", "2024-03-01"]
})

## 문자열 >>> 날짜형 변환
print("변환 전 타입 : ", df['나이'].dtype)

df['나이']  =  pd.to_numeric( df['나이'], downcast='unsigned')

print("변환 후 타입 : ", df['나이'].dtype)
```

◆ 자료형 변환

- 문자형 컬럼 → 날짜형 변환

pandas.**to_datetime(** 데이터 **)**

→ arg : 변환할 데이터. Series, 리스트, 배열, 컬럼 데이터

→ errors : 숫자로 변환할 수 없는 값이 있을 때 처리 설정
'raise' 오류 발생, 'coerce' 변환 불가 NaN 처리

→ format : 날짜 문자열 형식 지정

→ dayfirst : 일이 먼저 나오는 날짜 형식 처리

→ yearfirst : 연도가 먼저 나오는 날짜 형식 처리

→ utc : 시간대를 UTC 기준으로 변환

◆ 자료형 변환

- 문자형 컬럼 → 날짜형 변환

```
# 샘플 데이터 생성
df = pd.DataFrame({
    "이름": ["김민수", "이서연", "박지훈"],
    "나이": ["20", "21", "19"],
    "점수": ["85.5", "90.0", "78.5"],
    "가입일": ["2024-01-10", "2024-02-15", "2024-03-01"]
})

## 문자열 >>> 날짜형 변환
print("변환 전 타입 : ", df['가입일'].dtype)

df["가입일"] = pd.to_datetime(df["가입일"])

print("변환 후 타입 : ", df['가입일'].dtype)
```

◆ 범주형 데이터 인코딩

- **인코딩(Encoding)**

  → 문자형 또는 범주형 데이터 >>> **숫자 형태로 변환 작업**

  → 분석이나 머신러닝 모델이 처리할 수 있도록

❖ 메서드

연속형 => 범주형 pandas.**cut( )**

범주형 => 연속형 pandas.**get_dummies( )**

◆ 범주형 데이터 인코딩

- 연속형>>>범주형 : 이산화

연속형 데이터 일정 구간 ==> 범주형 데이터 변환

```
pandas.cut( x=df[컬럼명],                    # 데이터 배열
            bins=n,                          # 경계 값 리스트
            labels=bin_names,                # bin 이름
            include_lowest=True)             # 첫 경계값 포함
)
```

◆ 범주형 데이터 인코딩

- **범주형>>>연속형 : 인코딩**

❖ **라벨 인코딩**

알파벳 순서에 따라 문자형 데이터를 unique한 숫자형으로 매핑

ID	과일
1	사과
2	바나나
3	체리

LabelEncoder

ID	과일
1	0
2	1
3	2

◆ 범주형 데이터 인코딩

- **범주형>>>연속형 : 인코딩**

❖ **라벨 인코딩**

알파벳 순서에 따라 문자형 데이터를 unique한 숫자형으로 매핑

```
Series.map(
        arg,            # 값을 어떻게 바꿀지 변환 규칙. 보통 딕셔너리 사용
        na_action       # 결측값 처리 방식 지정
)
```

◆ 범주형 데이터 인코딩

- 범주형 >>> 연속형 : 인코딩

```
irisDF_map = {
    "Setosa": 0,
    "Versicolor": 1,
    "Virginica": 2
}

## 결측값 NaN에는 함수 적용하여 NaN
irisDF["variety_label"] = irisDF['variety'].map(irisDF_map)

## 결측값 NaN에는 함수를 적용하지 않음
irisDF["variety_label"] = irisDF['variety'].map(irisDF_map,
                                                na_action='ignore')
```

◆ 범주형 데이터 인코딩

- **범주형>>>연속형 : 인코딩**

❖ **원 핫 인코딩 (One Hot Encoding)**

라벨 수 만큼 자릿수 만든 해 해당 자리에 있으면 0, 없으면 1

ID	과일
1	사과
2	바나나
3	체리

One-Hot Encoding

ID	사과	바나나	체리
1	1	0	0
2	0	1	0
3	0	0	1

◆ 범주형 데이터 인코딩

- **범주형>>>연속형 : 인코딩**

❖ **원 핫 인코딩 (One Hot Encoding)**

라벨 수 만큼 자릿수 만든 해 해당 자리에 있으면 0, 없으면 1

```
pandas.get_dummies(
        data                    # 데이터프레임
        columns=[컬럼이름],     # 인코딩할 컬럼명/컬럼리스트
        prefix,                 # 생성될 컬럼 이름 앞에 붙일 접두어
        dtype = int             # 결과값을 0/1 정수로 변환
)
```

◆ 범주형 데이터 인코딩

- 범주형>>>연속형 : 인코딩

```
df_encoded = pd.get_dummies(
    df,
    columns=["variety"],
    dtype=int
)

print(df_encoded.head())
```

	sepal.length	sepal.width	petal.length	petal.width	variety_Setosa	variety_Versicolor	variety_Virginica
0	5.1	3.5	1.4	0.2	1	0	0
1	4.9	3.0	1.4	0.2	1	0	0
2	4.7	3.2	1.3	0.2	1	0	0
3	4.6	3.1	1.5	0.2	1	0	0
4	5.0	3.6	1.4	0.2	1	0	0

데이터 전처리 - 필터링

데이터 전처리

◆ 필터링(Filtering)

- Series, DataFrame에서 **특정 조건식 만족하는 원소만 추출**
- 대표적인 방법 ➔ **불린 인덱싱(Boolean Indexing)**

◆ 필터링(Filtering)

- 불린 인덱싱(Boolean Indexing)

  - 비교 연산자 or 논리 연산자 결과값 ➔ True/False
  - 값이 **bool 타입으로만 구성된 Series**
  - **같은 Index가 True인 행/열만 추출 ➔ 인덱스처럼 사용**
  - MASK라고도 함

◆ 필터링(Filtering)

- 불린 인덱싱(Boolean Indexing)

연산결과					
False					
False					
True					
False					
True					
True					
False					

◆ 필터링(Filtering)

- 불린 인덱싱(Boolean Indexing)

비교 연산자 → >, >=, <, <=, ==, !=

논리 연산자 → (왼쪽 결과) **&** (오른쪽 결과)

(왼쪽 결과) **|** (오른쪽 결과)

~ (조건식)

◆ 필터링(Filtering)

- 불린 인덱싱(Boolean Indexing)

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [2, 2, 2]})

## - 특정 행의 원소만
result1 = df.iloc[0]>=2
print(f'result1 ====₩n{result1}', end='₩n₩n')

## - 0번, 1번 행의 원소만
result2 = df.iloc[0:2]>=2
print(f'result2 ====₩n{result2}', end='₩n₩n')
```

```
result1 ====
A    False
B     True
Name: 0, dtype: bool
```

```
result2 ====
       A     B
0  False  True
1   True  True
```

◆ 필터링(Filtering)

- 특정 값 가진 데이터 추출

```
DataFrame[열].isin( 추출값 리스트 )
```

- 특정 값 제외한 데이터 추출

```
~ DataFrame[열].isin( 추출값 리스트 )
```

◆ 필터링(Filtering)

- 특정 값 가진 데이터 추출

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2, 3], 'B': ['a', 'b', 'f']})

print('df.isin()' , df.isin([1, 3, 12, 'a']) , end='₩n₩n')

print('~ df.isin()' , ~df.isin([1, 3, 12, 'a']) , end='₩n₩n')
```

```
df.isin()
      A     B
0  True  True
1 False False
2  True False
```

```
~ df.isin()
      A     B
0 False False
1  True  True
2 False  True
```

◆ 필터링(Filtering)

- query( )

  - 불리언 인덱싱으로 데이터 프레임의 각 행과 일치
  - True 로 평가되는 행만 반환
  - **특정 조건을 만족하는 결과를 추출**

◆ 필터링(Filtering)

- query( ) : 특정 값 가진 데이터 추출

```
df = pd.DataFrame({ 'A': range(1, 6),
                    'B': range(10, 0, -2),
                    'C C': range(10, 5, -1)})

result1 = df.query('A > B')
print(f'[result1]₩n{result1}', end='₩n₩n')
```

[df]

	A	B	C C
0	1	10	10
1	2	8	9
2	3	6	8
3	4	4	7
4	5	2	6

[result1]

	A	B	C C
4	5	2	6

◆ 필터링(Filtering)

- query( ) : 특정 값 가진 데이터 추출

```
df = pd.DataFrame({ 'A': range(1, 6),
                    'B': range(10, 0, -2),
                    'C C': range(10, 5, -1)})

result2 = df[df.A > df.B]
print(f'[result2]₩n{result2}', end='₩n₩n')


result3 = df[df.B == df['C C']]
print(f'[result3]₩n{result3}', end='₩n₩n')
```

```
[result2]
   A  B  C C
4  5  2    6
```

```
[result3]
   A   B  C C
0  1  10   10
```